

Project Title : AI-Enhanced SOA-Based Student Course Registration System

SOA Project Review-1



Under the Guidance of

Ms. Navyatha Ravi

Assistant Professor,

Department of Computer Science and Engineering,

KLH CSE Bowrampet Campus



Team Members

S.Trividha - 2420030010

K.Deepthi - 2420030484

Outline of Project Work

- Focus

- Introduction

- Literature Survey

- Challenges

- Research Objectives

- Conclusion & Future Work

- References

- Publications Details

Problem Statement

Traditional student course registration systems are typically built as monolithic applications where prerequisite validation, seat allocation, fee verification, and student notification are all handled within a single tightly coupled codebase.

This creates several recurring issues: any change to one module (e.g., a new prerequisite rule) risks breaking unrelated functionality; the system cannot scale a single high-demand function (like seat allocation during peak registration) independently of the rest; and integrating with other university systems (finance, timetabling, alumni records) requires custom point-to-point interfaces that are expensive to maintain.

There is a need for an architecture that decomposes registration into independent, reusable, and interoperable services that can evolve and scale on their own.

Objectives

- To design a Service-Oriented Architecture (SOA) for student course registration using SOAP/WSDL-based web services.
- To decompose registration functionality into independent services: prerequisite/conflict validation, seat allocation, and notification.
- To coordinate these services through an Enterprise Service Bus (ESB) using BPEL-style orchestration.
- To enable service discoverability using a UDDI-style registry, decoupling consumers from fixed service endpoints.
- To demonstrate the SOA principles of loose coupling, reusability, interoperability, and standardized contracts through a working prototype.

Literature Survey

Authors	Title of the Paper	Methodology	Inferences	Limitation/Future work
Chembe et al. [1]	An Implementation of an Online-Based Registration System in Tertiary Institutions (IJERT, 2023)	Develops a web-based course registration application using Service-Oriented Architecture, grouping system components into web services accessible by students, department members, faculty, and registry staff	Confirms SOA's practicality for multi-stakeholder university registration workflows	Limited detail on orchestration mechanism (ESB/BPEL) between the grouped services
Tao et al. [2]	Service Oriented Business Process Development Using BPEL: A Case Study (2025)	Designs multiple business processes and implements them using Web services technologies, focusing on adaptability and flexibility for integration with dynamically changing business environments	Validates BPEL as a practical orchestration layer for composing multiple independent services into one workflow	Integration between the BPEL engine and the underlying web service framework required manual type-mapping workarounds, indicating tooling friction in classic SOA stacks
Academia.edu meta-study [3]	Student Registration Systems (compiled research)	Surveys microservices-architecture approaches to student registration systems, evaluating maintainability under frequent policy changes	Suggests a broader industry shift from classic SOA/ESB toward microservices for registration systems	Doesn't directly compare classic SOA (SOAP/ESB/BPEL) against microservices on the same registration use case

Problem Gap Identification

- Existing SOA-based registration studies (Chembe et al.) demonstrate service decomposition but don't detail the **orchestration mechanism** — how multiple services are sequenced into one transaction (prerequisite check → seat allocation → notification).
- BPEL case studies (Tao et al.) prove orchestration works for general business processes, but none specifically model the **registration domain's dependency chain** (a failed seat allocation must roll back or bypass a completed prerequisite check).
- SOA testing research (Academia.edu meta-study) flags that loose coupling complicates end-to-end validation, but no reviewed work addresses this specifically for **time-sensitive, high-concurrency academic transactions** like registration.
- The literature shows a shift toward microservices, but a **direct classic-SOA-vs-modern-microservices comparison on the same registration use case** is missing; this project's classic SOA implementation can serve as a baseline for that comparison.

Innovation, Creativity, and Novelty

- **Composite orchestrated transaction:** Unlike prior work that treats services independently, this project models registration as a single BPEL-style orchestrated transaction with defined rollback/compensation behavior (e.g., if seat allocation fails after prerequisite validation succeeds, the orchestrator reverts state rather than leaving it inconsistent).
- **Dynamic service discovery:** Using a UDDI-style registry rather than hardcoded endpoints means services can be relocated, versioned, or replaced (e.g., swapping the Notification Service for an SMS-based one) without touching the consumer or the ESB configuration.
- **Domain-specific validation service:** The Prerequisite and Conflict Validation Service is designed as a reusable rule-engine-backed service — the same service could later be reused by academic advising tools, not just the registration portal, demonstrating true SOA reusability rather than one-off integration.
- **Bridges classic SOA and modern practice:** By deliberately implementing classic SOA (SOAP/WSDL/ESB/BPEL) instead of REST/microservices, the project produces a concrete artifact that can be benchmarked against microservices approaches — directly addressing the gap identified above.

Feasibility and Project Plan

Phase	Duration	Deliverable
Requirements & design	Week 1	Abstract, architecture diagram.
Service development	Week 2 - 3	PCVS, SAS, NS implemented as independent SOAP services with working WSDL
Orchestration layer	Week 4	ESB/orchestrator sequencing the three services (BPEL or scripted equivalent)
Registry & integration	Week 5	UDDI-style discovery wired in; end-to-end registration flow working
Testing & documentation	Week 6	Functional testing, report writing, final presentation